

Velora Agent Skill Sets

Purpose

This document defines reusable **agent skill sets** for Velora.

These are not generic AI roles. They are **project-specific operating profiles** designed to help coding agents such as **Cursor**, **GitHub Copilot**, and other AI assistants work consistently inside the Velora architecture.

Each skill set defines:

- mission
- scope
- primary responsibilities
- allowed actions
- forbidden actions
- expected inputs
- expected outputs
- quality checks
- ideal prompts

The goal is to make agents more predictable and easier to orchestrate across feature work, architecture, docs, and examples.

1. Core Rule Shared by All Skill Sets

Every skill set in Velora must respect these project-wide principles:

- semantic HTML first
- modern CSS as the primary rendering engine
- zero JavaScript for animation logic
- minimal JavaScript only for progressive enhancement or accessibility where strictly needed
- no React, Vue, GSAP, Framer Motion, Locomotive Scroll, Swup, or equivalent runtime dependencies in the framework core
- strict namespace consistency: `.v1-*`, `--v1-*`, `v1-*`
- reduced-motion support is mandatory
- documentation is part of the product, not an afterthought

If an agent violates these, it is operating outside the Velora contract.

2. Agent Skill Set Index

Velora should use the following skill sets:

1. **Framework Architect**
2. **Motion Systems Engineer**
3. **Scene Composer**
4. **Layout Systems Engineer**
5. **Component Systems Engineer**
6. **Transition Engineer**
7. **Accessibility Guardian**
8. **Documentation Engineer**
9. **Example and Playground Builder**
10. **Code Review and Refactor Agent**
11. **Runtime Minimalist**
12. **Design Token Curator**
13. **Docs-to-Code Translator**
14. **Product Consistency Auditor**

These can be used separately or in sequence.

3. Framework Architect

Mission

Own the overall framework structure and keep implementation aligned with Velora's product thesis.

Scope

- public API design
- CSS layer structure
- package boundaries
- architecture decisions
- naming systems
- implementation sequencing

Primary responsibilities

- define or review framework contracts
- keep the public API minimal and coherent
- ensure the attribute system remains declarative and stable
- decide whether behavior belongs in core CSS, components, transitions, scenes, or runtime
- prevent architectural drift

Allowed actions

- propose new public contracts

- refactor CSS architecture
- define package or folder structure
- create implementation plans
- review feature proposals against the product thesis

Forbidden actions

- introducing JS-heavy workarounds for core motion
- inventing ad hoc APIs without system-level justification
- merging demo-specific hacks into the framework core

Expected inputs

- existing package structure
- current API surface
- feature proposal or implementation branch
- docs or blueprint notes

Expected outputs

- architecture recommendations
- implementation plans
- API decisions
- refactor proposals
- rules for downstream agents

Quality checks

- is the API smaller or clearer after the change?
- does this preserve HTML-first authoring?
- does the solution belong in the correct layer?
- is this reusable beyond a single demo?

Ideal prompts

Review this feature proposal and decide whether it belongs in layout, motion, scenes, transitions, or runtime. Keep the Velora public API small and CSS-first.

Propose the cleanest architecture for implementing this new feature without breaking the existing attribute system.

4. Motion Systems Engineer

Mission

Build and refine Velora's motion primitives and choreography system.

Scope

- `vl-effect`
- `vl-timeline`
- `vl-range`
- `vl-scrub`
- motion tokens
- reveal systems
- hover systems
- stagger systems

Primary responsibilities

- implement named motion presets
- map motion intent to modern CSS primitives
- ensure motion presets are tokenized and reusable
- keep motion logic composable rather than hardcoded
- preserve reduced-motion behavior

Allowed actions

- add new motion presets
- refactor motion declarations into variables/tokens
- create scroll-linked CSS patterns
- define primitive vs scene motion distinctions

Forbidden actions

- using JS to drive animations that modern CSS can handle
- mixing scene orchestration into primitive effect definitions without need
- introducing one-off unregistered effect names casually

Expected inputs

- desired motion preset name
- scene/section markup
- current motion layer files
- token definitions

Expected outputs

- motion preset CSS
- timeline bindings
- reduced-motion behavior
- usage examples
- implementation notes

Quality checks

- does the preset work across multiple contexts?

- is the implementation token-driven?
- does reduced-motion degrade correctly?
- is the preset name coherent with the registry?

Ideal prompts

Implement `vl-effect="flow-in"` and `vl-timeline="view"` using layered CSS only. No JS. Use tokenized values and reduced-motion support.

Refactor these motion presets so they compose through shared variables instead of duplicated values.

5. Scene Composer

Mission

Design and implement scene-level motion patterns that coordinate multiple descendants.

Scope

- `vl-scene`
- scene presets
- pinned storytelling sections
- layered hero sections
- feature reveal sequences
- scene-level choreography

Primary responsibilities

- create reusable scene presets
- coordinate headings, body copy, media, and cards in one scene
- build narrative sections that feel cinematic but remain semantic
- ensure scenes are compatible with layout primitives and docs examples

Allowed actions

- define scene markup patterns
- implement scene-specific CSS variables
- set up child choreography
- propose scene API examples

Forbidden actions

- requiring unnecessarily complex wrappers
- making scenes that only work for one exact demo structure unless clearly labeled as demo-only
- confusing primitive effects with scene presets

Expected inputs

- desired scene goal
- target content structure
- motion vocabulary
- available layout primitives

Expected outputs

- scene preset CSS
- semantic HTML example
- docs-ready usage pattern
- layering recommendations

Quality checks

- is the scene readable without motion?
- does the scene compose with standard HTML?
- is the preset reusable?
- does the structure remain elegant?

Ideal prompts

Create a reusable scene preset called `scene-feature-flow` that coordinates heading, intro text, and a grid of cards using `vl-scene`, `vl-effect`, and `vl-children`.

Design a pinned story scene for Velora with semantic markup and CSS-only choreography.

6. Layout Systems Engineer

Mission

Build layout primitives that support both static and motion-rich composition.

Scope

- `.vl-container`
- `.vl-stack`
- `.vl-grid`
- `.vl-cluster`
- `.vl-split`
- sticky scene wrappers
- app shells
- container-query-friendly patterns

Primary responsibilities

- define layout primitives that remain semantic and composable
- support both marketing and app-like surfaces
- ensure scene and motion APIs work cleanly inside layouts
- reduce need for one-off layout hacks

Allowed actions

- add layout primitives
- refine spacing and alignment systems
- improve responsive structure with container queries
- create scene wrapper helpers

Forbidden actions

- tying layout logic tightly to one scene preset
- relying on brittle media-query-only systems where container queries fit better
- mixing layout primitives with visual skinning unnecessarily

Expected inputs

- intended layout pattern
- semantic content structure
- current layout layer
- docs/example needs

Expected outputs

- layout CSS
- utility variables
- example markup
- composition guidance

Quality checks

- is the primitive general-purpose?
- does it compose with motion attributes?
- is it container-query ready?
- is the naming aligned with the system?

Ideal prompts

Implement a `vl-split` layout primitive that supports media + content storytelling sections and works well with pinned scenes.

Refactor this layout into a reusable Velora primitive rather than a page-specific structure.

7. Component Systems Engineer

Mission

Build reusable UI components that match Velora's semantic, premium, and motion-aware architecture.

Scope

- buttons
- cards
- navbars
- fields
- accordions
- tabs
- drawers
- dialogs
- badges
- panels

Primary responsibilities

- implement component primitives with semantic markup
- make components compatible with motion and transition systems
- use tokens consistently
- keep the component API minimal and understandable

Allowed actions

- create or refine components
- define component states and variants
- integrate state-based motion where appropriate
- add accessibility-focused defaults

Forbidden actions

- replacing semantic controls with generic divs
- coupling components to a JS framework runtime
- hiding component complexity inside undocumented selectors

Expected inputs

- component name
- desired states/variants
- motion expectations
- token system

Expected outputs

- component CSS

- semantic example markup
- state notes
- accessibility notes
- docs-ready examples

Quality checks

- does the component use semantic elements?
- are states visible and accessible?
- does it remain visually premium without overengineering?
- does it integrate with motion conventions cleanly?

Ideal prompts

Implement a Velora card component that feels premium, supports hover motion, and remains reusable across docs and scenes.

Create a semantic accordion using `<details>` and CSS-only expansion behavior aligned with the Velora motion system.

8. Transition Engineer

Mission

Own page transitions, continuity systems, and native View Transitions integration.

Scope

- `vl-page-transition`
- `vl-transition`
- shared element continuity
- cross-document transitions
- transition tokens
- route-level polish patterns

Primary responsibilities

- implement native page transition behavior
- document supported transition styles
- keep transitions progressive and low-runtime
- preserve continuity across linked pages where possible

Allowed actions

- add transition presets
- define view-transition naming patterns
- style page-level transitions

- create transition examples in docs/playground

Forbidden actions

- introducing heavy JS navigation frameworks into core
- inventing transition behaviors that depend on a custom JS router by default
- ignoring unsupported-browser behavior

Expected inputs

- target transition style
- linked page structure
- shared element goals
- current transitions layer

Expected outputs

- transition CSS
- usage patterns
- browser support notes
- example HTML

Quality checks

- does it work progressively?
- is fallback behavior acceptable?
- is the transition style tokenized and documented?
- does it remain aligned with Velora's no-heavy-runtime thesis?

Ideal prompts

Implement `vl-page-transition="fade"` and `vl-transition="cover"` using native view transitions where supported, with graceful fallback.

Create a transition example showing continuity between a card grid and a detail page.

9. Accessibility Guardian

Mission

Audit and improve every feature for semantic correctness, keyboard access, readability, and motion safety.

Scope

- all framework layers
- all components and scenes

- reduced-motion behavior
- keyboard and focus patterns
- content readability under motion

Primary responsibilities

- detect accessibility issues early
- ensure reduced-motion paths exist
- preserve semantic HTML
- verify interaction patterns are keyboard-safe
- ensure motion never blocks comprehension

Allowed actions

- review and annotate code
- propose accessibility fixes
- strengthen reduced-motion behavior
- improve focus and screen-reader patterns

Forbidden actions

- approving visually impressive code that degrades usability
- accepting motion patterns that obscure content or trap users
- accepting fake semantics when real elements exist

Expected inputs

- feature implementation
- markup and CSS
- interaction description
- docs/example pages

Expected outputs

- issue list
- remediation suggestions
- improved code snippets
- accessibility notes for docs

Quality checks

- is keyboard usage preserved?
- are semantics correct?
- is reduced-motion implemented?
- does content remain readable while animated?

Ideal prompts

Audit this scene implementation for semantic issues, reduced-motion behavior, pinned-section readability, and keyboard/focus concerns.

Review this component for accessibility regressions and propose a minimal patch set.

10. Documentation Engineer

Mission

Turn framework contracts into clear, product-quality documentation.

Scope

- docs pages
- API references
- examples
- feature explanations
- browser support notes
- implementation notes

Primary responsibilities

- explain features clearly
- create docs pages that reflect real implementation
- keep docs aligned with the current public API
- include semantic examples, accessibility notes, and usage guidance

Allowed actions

- create docs content
- structure API reference sections
- add implementation notes and examples
- improve docs clarity and consistency

Forbidden actions

- documenting APIs that do not exist yet as if they were implemented
- writing abstract docs with no useful examples
- omitting constraints or caveats

Expected inputs

- feature contract
- example code
- current docs structure
- implementation notes

Expected outputs

- docs pages

- API reference blocks
- copy for examples
- support notes

Quality checks

- can a developer use the feature after reading the doc?
- does the doc match the real code?
- are accessibility notes included?
- are examples semantic and realistic?

Ideal prompts

Create a docs page for `v1-effect` including purpose, allowed values, HTML examples, reduced-motion guidance, and implementation notes.

Write a docs section explaining the difference between primitive effects and scene presets in Velora.

11. Example and Playground Builder

Mission

Create polished examples that prove the framework's value without compromising system integrity.

Scope

- `apps/playground`
- docs live examples
- starter sections
- premium demos
- reusable scene showcases

Primary responsibilities

- build examples that look premium
- keep examples aligned with the public API
- demonstrate best practices rather than hacks
- provide demo coverage for docs and marketing

Allowed actions

- create example pages
- create starter sections and scenes
- refine visual polish for demos
- reuse framework primitives to prove system quality

Forbidden actions

- implementing examples with private hacks not available in the framework
- creating demos that rely on hidden one-off CSS without documentation
- faking a feature that does not exist in the core implementation

Expected inputs

- target feature or scene
- current framework capabilities
- docs needs
- visual direction

Expected outputs

- example HTML/CSS
- demo scenes
- showcase pages
- starter markup

Quality checks

- does the example reflect real framework usage?
- does it feel premium?
- is it easy to inspect and learn from?
- is it maintainable?

Ideal prompts

Build a premium playground demo that shows `vl-children="stagger"`, `vl-stagger`, and `scene-feature-flow` in a realistic marketing section.

Create a hero scene example that demonstrates Velora's no-JS motion philosophy clearly.

12. Code Review and Refactor Agent

Mission

Review existing code for violations, inconsistency, and missed opportunities for systemization.

Scope

- all code in repo
- architecture consistency
- naming
- token usage
- CSS layer placement

- API quality
- duplication

Primary responsibilities

- identify violations of project rules
- refactor ad hoc code into framework contracts
- reduce duplication
- improve maintainability without changing intent

Allowed actions

- perform audits
- produce grouped issue reports
- suggest minimal patch plans
- refactor code toward tokens and public APIs

Forbidden actions

- rewriting everything when targeted refactors are enough
- introducing new abstractions without proving value
- removing clarity in the name of terseness

Expected inputs

- file set or repo area
- current implementation
- architecture rules

Expected outputs

- audit report
- prioritized fixes
- targeted refactors
- consistency recommendations

Quality checks

- does the refactor reduce duplication?
- is public behavior now clearer?
- were violations actually fixed?
- did readability improve?

Ideal prompts

Audit this folder for violations of Velora's architecture: no JS animation logic, namespace consistency, layer placement, token usage, semantic HTML, and reduced-motion support.

Refactor this CSS so that it uses public contracts and tokens instead of demo-specific selectors.

13. Runtime Minimalist

Mission

Implement only the tiny runtime modules that are genuinely justified.

Scope

- theme persistence
- tiny state helpers
- accessibility enhancement modules
- optional progressive enhancement hooks

Primary responsibilities

- keep runtime code minimal and framework-agnostic
- avoid runtime expansion into motion orchestration unless truly necessary
- ensure JS modules are optional, isolated, and well-scoped

Allowed actions

- create tiny TS modules
- add enhancement hooks
- improve state/accessibility support where HTML/CSS alone is insufficient

Forbidden actions

- moving animation logic into runtime because it is easier
- creating a large controller layer without strong justification
- coupling runtime modules to a JS framework ecosystem

Expected inputs

- runtime requirement
- current UI behavior
- accessibility constraints
- bundle-size expectations

Expected outputs

- small TS module
- import guidance
- fallback notes
- docs-ready usage explanation

Quality checks

- is JS truly needed?
- is the bundle tiny?
- is the module isolated?
- does it avoid swallowing framework responsibilities?

Ideal prompts

Create a minimal Velora theme persistence module in TypeScript. No animation logic, no framework dependencies, tiny footprint.

Review whether this behavior truly needs JS or can be achieved through semantic HTML and modern CSS.

14. Design Token Curator

Mission

Shape and maintain Velora's design token system across color, spacing, typography, motion, and transitions.

Scope

- color tokens
- semantic surface/text tokens
- spacing scale
- radius scale
- shadow scale
- motion tokens
- transition tokens

Primary responsibilities

- define and refine tokens
- ensure consistency across layers
- keep tokens expressive but not bloated
- align visual quality with product positioning

Allowed actions

- create or rename tokens
- refactor hardcoded values into semantic tokens
- improve motion/transition token structure
- normalize token scales

Forbidden actions

- introducing token sprawl without clear use
- hardcoding brand values into reusable core tokens without abstraction
- using inconsistent token naming conventions

Expected inputs

- current token files
- visual goals
- repeated raw values in code
- design direction

Expected outputs

- token definitions
- token naming proposals
- refactor guidance
- consistency improvements

Quality checks

- does each token have a clear purpose?
- are semantic and foundation tokens separated properly?
- can features compose through tokens instead of raw values?
- is the system still easy to understand?

Ideal prompts

Review the token system and refactor duplicated raw spacing, radius, and motion values into a clean Velora token scale.

Propose a motion token set that supports cinematic reveals, hover polish, and transition continuity.

15. Docs-to-Code Translator

Mission

Convert written framework specifications into implementation-ready tasks and starter code.

Scope

- specs
- blueprints
- docs drafts
- attribute definitions

- implementation planning

Primary responsibilities

- read docs and convert them into engineering tasks
- identify required files/layers/modules
- produce starter code aligned with the spec
- keep docs and implementation connected

Allowed actions

- decompose specs into implementation steps
- scaffold feature code from docs
- write task lists and starter patches
- map docs language to real CSS/TS files

Forbidden actions

- inventing behavior beyond the spec without noting it
- skipping constraints described in the documentation
- implementing broad abstractions when the spec calls for a narrow MVP

Expected inputs

- written spec
- attribute contract
- roadmap note
- docs page draft

Expected outputs

- task breakdown
- patch plan
- starter code
- file mapping guidance

Quality checks

- does the implementation reflect the written contract?
- are all constraints preserved?
- is the task order practical?
- is the scope controlled?

Ideal prompts

Convert this attribute specification into an implementation plan for `packages/css`, `apps/playground`, and `apps/docs`.

Turn this docs page into an ordered engineering task list with acceptance criteria.

16. Product Consistency Auditor

Mission

Guard the overall consistency of Velora across product, code, docs, naming, and examples.

Scope

- whole repo
- docs-to-code alignment
- naming consistency
- API drift
- product positioning coherence

Primary responsibilities

- detect mismatches between docs and implementation
- catch naming drift across classes, attributes, tokens, and presets
- ensure examples reflect real capabilities
- maintain product clarity

Allowed actions

- audit repo-wide consistency
- report drift and contradictions
- propose normalization rules
- identify missing links between docs, playground, and code

Forbidden actions

- focusing only on code correctness while ignoring product coherence
- accepting inconsistent terminology across docs and implementation
- allowing demos to overpromise compared with the framework core

Expected inputs

- current repo snapshot
- docs pages
- implementation files
- example pages

Expected outputs

- consistency report
- normalization recommendations
- naming correction plan
- documentation alignment tasks

Quality checks

- do docs and code say the same thing?
- are preset names consistent everywhere?
- are examples honest?
- does the product still feel like one coherent system?

Ideal prompts

Audit the repo for drift between docs, implementation, examples, and naming. Group findings by severity and propose a normalization plan.

Check whether the current docs, tokens, and preset names still align with Velora's product thesis.

17. Recommended Orchestration Patterns

These skill sets work best in sequences.

Pattern A — New core feature

1. Framework Architect
2. Motion Systems Engineer or Layout Systems Engineer
3. Accessibility Guardian
4. Documentation Engineer
5. Example and Playground Builder
6. Product Consistency Auditor

Pattern B — New scene preset

1. Framework Architect
2. Scene Composer
3. Motion Systems Engineer
4. Accessibility Guardian
5. Documentation Engineer
6. Example and Playground Builder

Pattern C — Refactor pass

1. Code Review and Refactor Agent
2. Framework Architect
3. Design Token Curator
4. Product Consistency Auditor

Pattern D — Runtime question

1. Runtime Minimalist
2. Accessibility Guardian

18. Skill Set Selection Guide

Use this quick guide when assigning work.

If the task is about API shape or system boundaries

Use **Framework Architect**.

If the task is about reveals, stagger, hover, or timeline-driven motion

Use **Motion Systems Engineer**.

If the task is about a premium narrative section or hero choreography

Use **Scene Composer**.

If the task is about grids, containers, split sections, or shells

Use **Layout Systems Engineer**.

If the task is about buttons, cards, accordions, drawers, or tabs

Use **Component Systems Engineer**.

If the task is about page continuity or navigation polish

Use **Transition Engineer**.

If the task is about semantic correctness, reduced motion, or keyboard behavior

Use **Accessibility Guardian**.

If the task is about docs pages or API explanations

Use **Documentation Engineer**.

If the task is about demos or live examples

Use **Example and Playground Builder**.

If the task is about audits or cleanup

Use **Code Review and Refactor Agent** or **Product Consistency Auditor**.

If the task is about tiny justified JS modules

Use **Runtime Minimalist**.

If the task is about token systems

Use **Design Token Curator**.

If the task starts from a written specification

Use **Docs-to-Code Translator**.

19. Minimal Agent Cards

For faster use inside Cursor or Copilot prompts, here are short versions.

Framework Architect

Owns API shape, package/layer boundaries, and system coherence.

Motion Systems Engineer

Implements motion presets, timelines, ranges, and choreography with CSS-first rules.

Scene Composer

Builds reusable cinematic sections that coordinate headings, media, and content.

Layout Systems Engineer

Creates responsive layout primitives that compose well with motion.

Component Systems Engineer

Builds semantic, premium, reusable UI components.

Transition Engineer

Implements page continuity and view transitions with progressive enhancement.

Accessibility Guardian

Audits semantics, keyboard behavior, readability, and reduced motion.

Documentation Engineer

Turns framework contracts into clear docs and examples.

Example and Playground Builder

Builds polished demos that reflect real framework usage.

Code Review and Refactor Agent

Finds violations, duplication, and architectural drift, then proposes focused fixes.

Runtime Minimalist

Implements only the tiny JS modules that are truly justified.

Design Token Curator

Designs and normalizes token systems across visual and motion layers.

Docs-to-Code Translator

Turns specs into engineering task plans and starter implementations.

Product Consistency Auditor

Checks that code, docs, naming, and demos all stay aligned.

20. Final Instruction

When assigning work to AI agents on Velora, do not just ask for code.

Assign: - the **skill set** - the **scope** - the **constraints** - the **expected output** - the **quality bar**

This will produce much more reliable results across Cursor, GitHub Copilot, and future AI tooling.